Building a Secure Private Browser

A comprehensive guide to constructing an anti-tracking, fingerprint-resistant, anonymous web browser using Python and PyQt5.

> **Overview:** This document compiles the step-by-step process of building a custom web browser from scratch. We evolve from a basic "Incognito Mode" shell to a hardened privacy tool featuring User-Agent spoofing, Encrypted DNS (DoH), domain-level ad blocking, and Tor network routing.

Phase 1: Prerequisites & Environment Setup

To build the interface and access the underlying Chromium browser engine, we use `PyQt5` and its WebEngine component. Open your terminal or command prompt and install the dependencies:

```
pip install PyQt5 PyQtWebEngine
```

Phase 2: The Core Concepts of Browser Privacy

A standard browser leaves massive trails of data. To make our browser truly private, we addressed multiple vectors of tracking throughout our build:

- **Local Privacy:** Preventing the browser from saving history and cookies to your hard drive. We achieve this by forcing a Memory-Only Cache and disabling persistent cookies.
- **Fingerprinting:** Websites track you based on your unique hardware and software combination. We mitigate this by Spoofing the User-Agent to look like a standard Windows Chrome user.
- **DNS Leaks:** By default, your Internet Service Provider can see your DNS requests in plain text. We encrypt these using DNS-over-HTTPS (DoH) via Cloudflare.
- **Third-Party Trackers:** Websites load invisible tracking scripts. We built a network interceptor to block known malicious domains.
- **IP Exposure:** Your real IP address is visible to websites. We route our traffic through the Tor network via a SOCKS5 proxy to anonymize our connection.

Phase 3: Building the Network Bouncer (AdBlocker)

Instead of hardcoding a few blocked words, our browser uses a high-speed Python `set` to check every network request against a massive list of known tracking domains.

To enable this, create a text file named `blocklist.txt` in the same folder as your Python script. You can populate this file with thousands of domains from community lists like the StevenBlack Unified Hosts repository.

Phase 4: The Final Integrated Code

Below is the complete, integrated code for the final product. Save this file as `private_browser.py`.

```
import sys
from PyQt5.QtCore import *
from PyQt5.QtWidgets import *
from PyQt5.QtWebEngineWidgets import *
from PyQt5.QtNetwork import QNetworkProxy

# -------------------------------------------------------
# 1. ENCRYPTED DNS (DoH) CONFIGURATION
# -------------------------------------------------------
# Enable DNS-over-HTTPS via Cloudflare before QApplication starts
sys.argv.append("--enable-features=dns-over-https")
cloudflare_doh_url = "https%3A%2F%2Fcloudflare-dns.com%2Fdns-query"
sys.argv.append(f"--force-fieldtrial-params=DnsOverHttps.Enabled:server/
{cloudflare_doh_url}")

# -------------------------------------------------------
# 2. THE REQUEST INTERCEPTOR (AD & TRACKER BLOCKER)
# -------------------------------------------------------
class TrackerBlocker(QWebEngineUrlRequestInterceptor):
    def __init__(self):
        super().__init__()
        self.blocked_domains = set()
        self.load_blocklist()

    def load_blocklist(self):
        try:
            with open("blocklist.txt", "r") as file:
                for line in file:
                    domain = line.strip()
                    if domain and not domain.startswith("#"):
                        # Grabs the domain even if the list has IP prefixes
                        clean_domain = domain.split()[-1]
                        self.blocked_domains.add(clean_domain)
            print(f"Loaded {len(self.blocked_domains)} domains into the blocklist.")
        except FileNotFoundError:
            print("Warning: blocklist.txt not found. Operating without ad blocking.")

    def interceptRequest(self, info):
        url = info.requestUrl()
        host = url.host()
        parts = host.split('.')

        # Check domain and subdomains against the blocklist
        for i in range(len(parts) - 1):
            parent_domain = '.'.join(parts[i:])
            if parent_domain in self.blocked_domains:
                print(f"Blocked: {host}")
```

```
                info.block(True)
                return

# ------------------------------------------------------------
# 3. THE MAIN BROWSER APPLICATION
# ------------------------------------------------------------
class PrivateBrowser(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("Shield - Private Browser")
        self.resize(1200, 800)

        # --- NETWORK ROUTING (TOR INTEGRATION) ---
        # Note: Requires the Tor service to be running locally on port 9050
        proxy = QNetworkProxy()
        proxy.setType(QNetworkProxy.Socks5Proxy)
        proxy.setHostName("127.0.0.1")
        proxy.setPort(9050)
        QNetworkProxy.setApplicationProxy(proxy)

        # --- PRIVACY PROFILE ---
        self.profile = QWebEngineProfile()

        # Spoof User-Agent to blend in
        chrome_ua = "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
(KHTML, like Gecko) Chrome/120.0.0.0 Safari/537.36"
        self.profile.setHttpUserAgent(chrome_ua)

        # Attach Tracker Blocker
        self.interceptor = TrackerBlocker()
        self.profile.setUrlRequestInterceptor(self.interceptor)

        # Force Memory-Only Cache and No Persistent Cookies
        self.profile.setPersistentCookiesPolicy(QWebEngineProfile.NoPersistentCookies)
        self.profile.setHttpCacheType(QWebEngineProfile.MemoryHttpCache)

        # --- UI SETUP ---
        self.webpage = QWebEnginePage(self.profile, self)
        self.browser = QWebEngineView()
        self.browser.setPage(self.webpage)
        self.browser.setUrl(QUrl('https://duckduckgo.com'))
        self.setCentralWidget(self.browser)

        # Navigation Bar
        navbar = QToolBar()
        self.addToolBar(navbar)
```

```
        back_btn = QAction('Back', self)
        back_btn.triggered.connect(self.browser.back)
        navbar.addAction(back_btn)

        forward_btn = QAction('Forward', self)
        forward_btn.triggered.connect(self.browser.forward)
        navbar.addAction(forward_btn)

        reload_btn = QAction('Reload', self)
        reload_btn.triggered.connect(self.browser.reload)
        navbar.addAction(reload_btn)

        self.url_bar = QLineEdit()
        self.url_bar.returnPressed.connect(self.navigate_to_url)
        navbar.addWidget(self.url_bar)
        self.browser.urlChanged.connect(self.update_url)

    def navigate_to_url(self):
        url = self.url_bar.text()
        if not url.startswith('http'):
            url = 'https://' + url
        self.browser.setUrl(QUrl(url))

    def update_url(self, q):
        self.url_bar.setText(q.toString())

# ------------------------------------------------------------
# 4. EXECUTION
# ------------------------------------------------------------
if __name__ == '__main__':
    app = QApplication(sys.argv)
    window = PrivateBrowser()
    window.show()
    sys.exit(app.exec_())
```

Phase 5: Compiling into a Standalone Executable

To turn this Python script into a fully functional desktop application (like an `.exe`) that you can run without opening a terminal, we use **PyInstaller**.

1. Install PyInstaller:
   `pip install pyinstaller`

2. Compile the script (hiding the background terminal with the `--noconsole` flag). On Windows, use this module execution approach to bypass PATH issues:
   `python -m PyInstaller --onefile --noconsole private_browser.py`

> **Important Deployment Step:** Once PyInstaller finishes, your standalone application will be located inside the newly created `dist` folder. If you move this application to your Desktop, you **must** also move a copy of your `blocklist.txt` file next to it. Without the text file, the app will run, but the adblocker will be bypassed.